

VentureMiner AI

MCP Architecture

Document: 07-AI-Architecture / 12_mcp_architecture

Version: v1.0

Date: 2026-07-20

Author: VentureMiner AI Documentation Team

Status: Approved

Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Document 12 — MCP Architecture

Table of Contents

1. Purpose & Scope
2. Why MCP
3. Architecture
4. Tool manifest
5. Server lifecycle
6. Authentication
7. Policy enforcement
8. Rate limiting & cost
9. Observability
10. Failure modes
11. Public MCP gateway (v2)
12. Appendix
 - 12.1 Glossary
 - 12.2 Revision history
 - 12.3 Cross-references

Document 12 — MCP Architecture

The Model Context Protocol (MCP) is the uniform tool surface for every agent. This document defines the gateway, the tool manifest, the policy enforcement, and the relationship with plugins.

Table of Contents

1. Purpose & Scope
2. Why MCP
3. Architecture
4. Tool manifest
5. Server lifecycle
6. Authentication
7. Policy enforcement
8. Rate limiting & cost
9. Observability
10. Failure modes
11. Public MCP gateway
12. Appendix

1. Purpose & Scope

This document is the contract for the MCP gateway — the boundary between agents and the rest of the world. Every tool call passes through it; every policy decision is made here.

2. Why MCP

- Uniform interface — every tool looks the same to an agent.
- Vendor-neutral — the protocol is open; we can swap providers.
- Policy enforcement — a single chokepoint for authz and rate limits.
- Future-proof — third-party MCP servers can plug in (v2).

3. Architecture

```
agent ■■■ MCP client (in-process) ■■■ MCP gateway (singleton per agent) ■■■ tool
server (HTTP/stdio)
```

- MCP client is embedded in the agent runtime; it speaks the MCP protocol.
- MCP gateway is a per-agent logical instance that:
 - Resolves tool names to servers.
 - Enforces authz (per workspace, per user, per tool).

4. Tool manifest

Every tool has a manifest:

```
id: T-MARKET-DATA-FETCHER
name: Market Data Fetcher
version: 1.2.0
description: Fetches market size and growth from a paid data provider.
risk_level: low # low | medium | high
pii_risk: false
input_schema: { type: object, properties: { query: { type: string } } }
output_schema: { type: object, properties: { ... } }
auth: { type: api_key, secret_ref: provider/marketdata/api_key }
cost: { per_call_usd: 0.02, weight: 1 }
rate_limit: { per_minute: 60, per_hour: 1000 }
timeout_ms: 5000
retry: { max: 2, backoff: exponential }
owner: ai-platform
```

The manifest is the source of truth; agents cannot call a tool without a registered manifest.

5. Server lifecycle

- Registration: a tool is registered by an admin in the plugin registry (Document 13).
- Activation: once active, the gateway can route to it.
- Deprecation: manifest version is marked deprecated; runs stop calling it after a grace period.
- Retirement: manifest removed; tool is no longer callable.

6. Authentication

- The gateway authenticates the agent's identity (via the user/workspace token attached to the run).
- The gateway authenticates the server's identity (mTLS or signed JWT).
- The user's authz is consulted for every call: does this user have permission to call this tool on this resource?

7. Policy enforcement

- Per-tool allow/deny per workspace (admin can disable).
- Per-resource scope (e.g. "this token can only call opportunity:read").
- Per-call policy (e.g. "no PII to external APIs", "no calls to non-allow-listed domains").

8. Rate limiting & cost

- Enforcement: gateway returns 429 or 429 + retry_after; the agent decides whether to retry, fall back, or surface.

9. Observability

- Every call is an OpenTelemetry span: mcp.call { tool, version, latency, cost, status }.
- Every call's input/output is logged (PII-redacted) at debug level.
- Cost is attributed to the workspace and surfaced in the admin dashboard.

10. Failure modes

Tool server timeout	Retry per manifest; if exceeded, fail the call.
Tool server 5xx	Retry with backoff; if exceeded, fail.
Auth failure	Reject; surface clear error.
Policy violation	Reject; audit log.
Rate limit	429; agent backs off.
Cost budget exceeded	Reject; orchestrator degrades.
Tool schema mismatch	Reject; alert.
Manifest corrupt	Reject; alert.

11. Public MCP gateway (v2)

- A public-facing MCP gateway exposes a curated subset of tools to third-party MCP-aware clients.
- Same gateway code path; different auth (per-client API token).
- Behind a separate rate limit and per-token budget.

12. Appendix

12.1 Glossary

MCP	Model Context Protocol
Tool manifest	The declarative description of a tool
Gateway	The chokepoint that enforces policy and routing
Risk level	A declared classification: low / medium / high

12.2 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

12.3 Cross-references

- Multi-Agent: Document 08.
- Plugin Architecture: Document 13.
- RAG: Document 10.

End of Document 12 — MCP Architecture. The gateway is the single chokepoint between agents and the world.